

ES 324: Signals and Systems

Complex Engineering Problem (CEP)

Design of High-Resolution Sampling System using Time-Interleaved
ADCs

Abdul Rafay Khan (Reg.no: 2023589)

Ahsan Ahmed (Reg.no: 2023089)

Submittied to:

Dr. Omer bin Saeed

December 23, 2025

1 Objective

The objective of this project is to design and simulate a sampling system capable of detecting a continuous-time target signal with a time resolution of 0.01 s (hundredth of a second). The design is constrained by the availability of samplers that operate at a maximum rate of 10 Hz ($T_s = 0.1$ s). A standard 10 Hz sampler is insufficient for this task as it cannot detect inter-sample events (e.g., a signal at $t = 5.89$ s).

2 Proposed System Design

2.1 Theoretical Approach: Time-Interleaved Sampling

To overcome the sampling rate limitation, we employ a **Time-Interleaved Analog-to-Digital Converter (TI-ADC)** architecture. This method uses K parallel low-speed samplers to achieve a higher aggregate effective sampling rate (F_{eff}).

Given:

- Base Sampler Rate (F_{low}): 10 Hz
- Required Time Resolution: 0.01 s (equivalent to $F_{req} = 100$ Hz)

The number of required parallel branches (K) is calculated as:

$$K = \frac{F_{req}}{F_{low}} = \frac{100 \text{ Hz}}{10 \text{ Hz}} = 10 \text{ branches} \quad (1)$$

2.2 System Operation

The input signal $x(t)$ is distributed to 10 parallel branches. Each branch k (where $k = 0, 1, \dots, 9$) contains:

1. **A Delay Element:** A fixed time delay τ_k defined by:

$$\tau_k = k \times 0.01 \text{ s} \quad (2)$$

2. **A Sampler:** A standard 10 Hz sampler.

The delays ensure that each sampler captures a different “slice” of time:

- Branch 0 samples at $t = 0.0, 0.1, \dots$
- Branch 1 samples at $t = 0.01, 0.11, \dots$
- ...
- Branch 9 samples at $t = 0.09, 0.19, \dots$

The outputs of all 10 branches are summed (multiplexed) to reconstruct a single discrete-time sequence $y[n]$ with an effective rate of 100 Hz.

2.3 Functional Block Diagram

The functional block diagram of the proposed system is shown below.

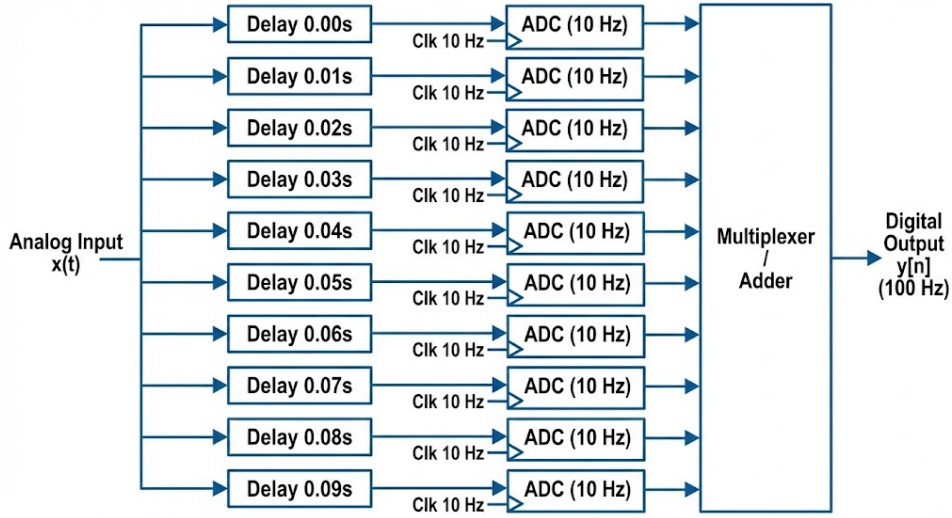


Figure 1: Functional Block Diagram of Time-Interleaved ADC System ($K = 10$)

2.4 Mathematical Verification

For a target signal occurring at $t_{target} = 5.89$ s:

- **Standard Sampler (Branch 0):** Samples at $t = 5.8$ s and $t = 5.9$ s. The impulse at 5.89 s falls in the dead zone.
Result: Signal Missed.
- **Branch 9 (Delay $\tau = 0.09$ s):** The sampler triggers at its nominal time $t_{nom} = 5.8$ s. However, due to the delay, it captures the signal value at:

$$t_{capture} = t_{nom} + \tau_9 = 5.8 \text{ s} + 0.09 \text{ s} = 5.89 \text{ s} \quad (3)$$

Since $t_{capture}$ matches t_{target} , the impulse is successfully recorded.

Result: Signal Detected.

3 Simulation Results

The proposed design was simulated using Python (see Annexure for code).

- **Continuous Input:**

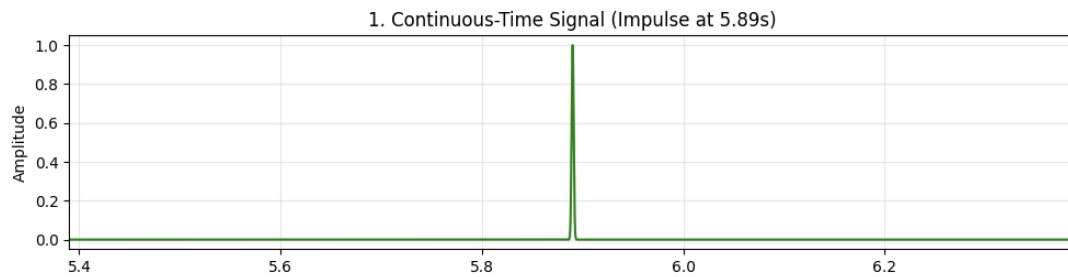


Figure 2: The plot shows the generated Gaussian impulse clearly located at 5.89 s.

- **Original Discrete (Single Sampler):**

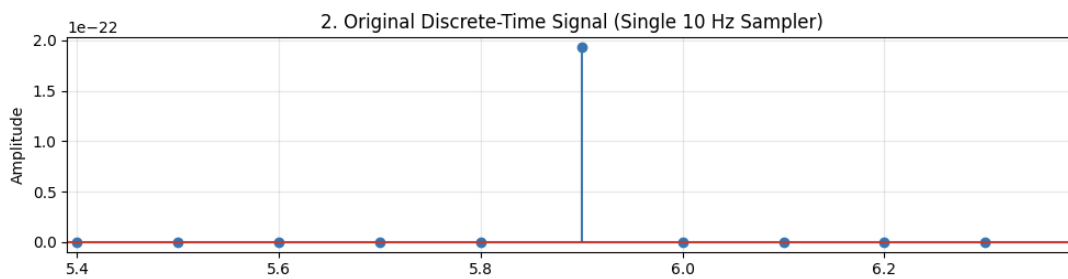


Figure 3: The plot shows zero amplitude at the target time, confirming that a single 10 Hz device fails to detect the signal.

- **Final Resampled Signal:**

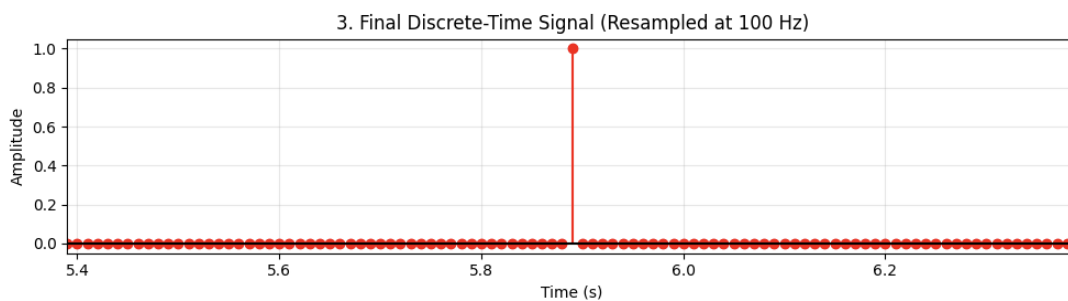


Figure 4: The output plot shows a distinct impulse at 5.89 s, proving the 10-branch system successfully reconstructed the signal.

4 Conclusion

This project successfully demonstrated the use of a Time-Interleaved ADC architecture to increase the effective sampling rate of a system. By using ten 10 Hz samplers with precise delays, we achieved a 100 Hz resolution, allowing for the detection of signal events at the hundredth-second level (e.g., 5.89 s) that were previously undetectable.

Annexure A: Python Simulation Code

The following Python code was used to simulate the system and verify the results for ID ending in 589.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # --- 1. User Configuration (Task 1 & 2) ---
5 # ID: 589 -> Target Time: 5.89 seconds
6 last_three_digits = "589"
7
8 sec = int(last_three_digits[0])
9 fract = int(last_three_digits[1:])
10 target_time = sec + fract/100.0
11
12 print(f"Target Signal Location: {target_time} seconds")
13
14 # --- 2. System Parameters ---
15 t_end = 10.0
16 fs_cont = 10000          # High-res simulation
17 t = np.linspace(0, t_end, int(fs_cont * t_end) + 1)
18
19 # Base Sampler
20 Fs_low = 10
21 Ts_low = 1/Fs_low
22
23 # Target resolution 100 Hz
24 Fs_req = 100
25 Ts_req = 1/Fs_req
26
27 # Calculate number of branches required (K)
28 K = int(Fs_req / Fs_low)
29
30 # --- 3. Generate Continuous Input Signal ---
31 sigma = 0.001
32 x_cont = np.exp(-0.5 * ((t - target_time)/sigma)**2)
33
34 # --- 4. System Simulation (Delays + Sampling) ---
35 N_high = int(np.floor(t_end * Fs_req)) + 1
36 y_final = np.zeros(N_high)
37 t_final = np.arange(N_high) / Fs_req
38
39 t_samples_base = np.arange(0, int(np.floor(t_end * Fs_low))+1) * Ts_low
40
41 for k in range(K):
42     delay = k * Ts_req
43     branch_sample_times = t_samples_base + delay
44
45     # Simulate sampling
46     branch_values = np.interp(branch_sample_times, t, x_cont, left=0.0,
47                               right=0.0)
48
49     # Interleaver logic
50     for i, val in enumerate(branch_values):
51         idx_high = i * K + k
52         if idx_high < N_high:
53             y_final[idx_high] = val

```

```
53
54 # --- 5. Generate Plots ---
55 zoom_window = 0.5
56 plt.figure(figsize=(10, 8))
57
58 # Plot A
59 plt.subplot(3, 1, 1)
60 plt.plot(t, x_cont, 'g')
61 plt.title(f'1. Continuous-Time Signal (Impulse at {target_time}s)')
62 plt.xlim(max(0, target_time - zoom_window), min(t_end, target_time +
        zoom_window))
63 plt.ylabel('Amplitude'); plt.grid(True, alpha=0.3)
64
65 # Plot B
66 x_single = np.interp(t_samples_base, t, x_cont, left=0.0, right=0.0)
67 plt.subplot(3, 1, 2)
68 plt.stem(t_samples_base, x_single)
69 plt.title('2. Original Discrete-Time Signal (Single 10 Hz Sampler)')
70 plt.xlim(max(0, target_time - zoom_window), min(t_end, target_time +
        zoom_window))
71 plt.ylabel('Amplitude'); plt.grid(True, alpha=0.3)
72
73 # Plot C
74 plt.subplot(3, 1, 3)
75 plt.stem(t_final, y_final, linefmt='r-', markerfmt='ro', basefmt='k-')
76 plt.title('3. Final Resampled Signal (100 Hz)')
77 plt.xlim(max(0, target_time - zoom_window), min(t_end, target_time +
        zoom_window))
78 plt.xlabel('Time (s)'); plt.ylabel('Amplitude'); plt.grid(True, alpha
        =0.3)
79
80 plt.tight_layout()
81 plt.show()
```

Listing 1: Python Code for Time-Interleaved ADC Simulation

Annexure B: Interactive Simulation Code

This additional script was developed to dynamically verify the system under various scenarios (changing target times, number of branches K , and pulse width) using interactive sliders.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from ipywidgets import interact, FloatSlider, IntSlider
4
5 def simulate_interleaved_system(target_time, K, sigma):
6     # --- System Parameters ---
7     # Handout Requirement: "Time range of 10 seconds"
8     t_end = 10.0
9     fs_cont = 10000          # High-res simulation
10    t = np.linspace(0, t_end, int(fs_cont * t_end) + 1)
11
12    # Base Sampler (10 Hz)
13    Fs_low = 10
14    Ts_low = 1/Fs_low
15
16    # Effective High-Speed Rate
17    Fs_req = Fs_low * K
18    Ts_req = 1/Fs_req
19
20    # --- 1. Generate Continuous Input ---
21    x_cont = np.exp(-0.5 * ((t - target_time)/sigma)**2)
22    if np.max(x_cont) > 0:
23        x_cont = x_cont / np.max(x_cont)
24
25    # --- 2. System Simulation ---
26    N_high = int(np.floor(t_end * Fs_req)) + 1
27    y_final = np.zeros(N_high)
28    t_final = np.arange(N_high) / Fs_req
29
30    t_samples_base = np.arange(0, int(np.floor(t_end * Fs_low))+1) *
    Ts_low
31
32    for k in range(K):
33        delay = k * Ts_req
34        branch_sample_times = t_samples_base + delay
35        branch_values = np.interp(branch_sample_times, t, x_cont, left
    =0.0, right=0.0)
36
37        for i, val in enumerate(branch_values):
38            idx_high = i * K + k
39            if idx_high < N_high:
40                y_final[idx_high] = val
41
42    # --- 3. Plotting ---
43    zoom_window = 0.5
44    fig, (ax1, ax2, ax3) = plt.subplots(3, 1, figsize=(10, 8))
45
46    # Plot A: Continuous
47    ax1.plot(t, x_cont, 'g')
48    ax1.set_title(f'Continuous Input (Target: {target_time:.2f}s)')
49    ax1.set_xlim(max(0, target_time - zoom_window), min(t_end,
    target_time + zoom_window))

```



```

50     ax1.grid(True, alpha=0.3)
51
52     # Plot B: Single Sampler
53     x_single = np.interp(t_samples_base, t, x_cont, left=0.0, right
54                           =0.0)
55     ax2.stem(t_samples_base, x_single)
56     ax2.set_title('Single 10 Hz Sampler (Often Misses Signal)')
57     ax2.set_xlim(max(0, target_time - zoom_window), min(t_end,
58                  target_time + zoom_window))
59     ax2.grid(True, alpha=0.3)
60
61     # Plot C: Final Output
62     ax3.stem(t_final, y_final, linefmt='r-', markerfmt='ro', basefmt='k
63              -')
64     ax3.set_title(f'Final Output (Effective Rate: {Fs_req} Hz with K={K
65                  } branches)')
66     ax3.set_xlim(max(0, target_time - zoom_window), min(t_end,
67                  target_time + zoom_window))
68     ax3.grid(True, alpha=0.3)
69
70     plt.tight_layout()
71     plt.show()
72
73 # --- Interactive Sliders ---
74 interact(simulate_interleaved_system,
75          target_time=FloatSlider(min=0.1, max=9.9, step=0.01, value
76                                  =5.89),
77          K=IntSlider(min=1, max=20, step=1, value=10),
78          sigma=FloatSlider(min=0.001, max=0.02, step=0.001, value
79                             =0.001)
80          );

```

Listing 2: Interactive Python Code for Scenario Verification